Activity 02 - Draft an ML Architecture Decision Record

Course: Google Professional Machine Learning Engineer Training Course code: C740 Version: v2.0 | 12 September 2026 Evidence: activity artifacts

Objective

Convert business volume, latency, security and explainability needs into a service decision.

Deliverables

- requirements.csv
- architecture_decision.py
- expected_decision.json
- A completed evidence/ folder with decision notes and verification output

Prerequisites

- Python 3.11 or later and a terminal; Docker only where the activity explicitly uses it.
- For optional live Google Cloud steps: the safe training project created in Activity 01.
- Start in this activity folder so relative paths and cleanup commands remain scoped.

Safety, cost and privacy

- Use a training project that contains no production or personal data.
- Replace every placeholder such as <PROJECT_ID> and <REGION> before a cloud command.
- Do not paste API keys or service-account JSON into notebooks, scripts or screenshots.
- Prefer Cloud Shell and Application Default Credentials. Never commit .env or credentials.
- Verify the result against the acceptance checklist, then run the cleanup section.

Evidence to retain

Create an evidence/ folder locally. Save command output, job or revision IDs, metric tables, screenshots without secrets, and a short decision note. Use filenames that include the activity number and date. Evidence is part of the learning outcome, not an optional extra.

Detailed step-by-step guide

Step 1 - Open requirements.csv and identify the business volume, freshness, latency, availability, privacy and explainability requirements

Open requirements.csv and identify the business volume, freshness, latency, availability, privacy and explainability requirements.

Check: Record the observable result or explain the error before continuing.

Step 2 - Run the architecture evaluator to score BigQuery ML, AutoML and custom training options

Run the architecture evaluator to score BigQuery ML, AutoML and custom training options.

```
python3 architecture_decision.py --requirements requirements.csv --out decision.json
```

Check: Record the observable result or explain the error before continuing.

Step 3 - Compare decision.json with expected_decision.json; investigate any difference instead of overwriting it

Compare decision.json with expected_decision.json; investigate any difference instead of overwriting it.

```
python3 -m json.tool decision.json
```

Check: Record the observable result or explain the error before continuing.

Step 4 - Draft an Architecture Decision Record with context, options, measurable drivers, selected service, rejected options and consequences

Draft an Architecture Decision Record with context, options, measurable drivers, selected service, rejected options and consequences.

Check: Record the observable result or explain the error before continuing.

Step 5 - Add a capacity calculation for peak QPS, 30% headroom and data growth over two years

Add a capacity calculation for peak QPS, 30% headroom and data growth over two years.

Check: Record the observable result or explain the error before continuing.

Step 6 - Add security controls for identity, encryption, network path, PII and audit logging

Add security controls for identity, encryption, network path, PII and audit logging.

Check: Record the observable result or explain the error before continuing.

Step 7 - Add testable acceptance criteria: p95 latency, quality floor, error rate, cost ceiling and rollback time

Add testable acceptance criteria: p95 latency, quality floor, error rate, cost ceiling and rollback time.

Check: Record the observable result or explain the error before continuing.

Step 8 - Peer-review the decision against A1 and A2 and save the review notes

Peer-review the decision against A1 and A2 and save the review notes.

Check: Record the observable result or explain the error before continuing.

Acceptance checklist

[ ] Every required artifact exists and opens without error.

[ ] Results are tied to the supplied mock data or an approved training project.

[ ] The conclusion cites metrics, IDs or command output rather than opinion alone.

[ ] No credentials, tokens, personal data or unredacted sensitive prompts are present.

[ ] Cleanup is completed or the retained resource is documented with an owner and expiry.

Troubleshooting

Symptom	Likely cause	Corrective action
403 PERMISSION_DENIED	API disabled, wrong project, or IAM role missing	Verify gcloud config get-value project, API state and the least-privilege role.
429 RESOURCE_EXHAUSTED	Quota or rate limit reached	Stop retries, inspect quota, add exponential backoff, and ask the trainer before increasing quota.
Region/location mismatch	Dataset, bucket, model or endpoint uses another location	Recreate only the training resource in one approved region; do not copy sensitive data casually.
Result differs from expected	Data, dependency or configuration version drifted	Compare file hashes, pinned versions and parameters; retain the diff as evidence.

Cleanup

Stop local processes and remove only resources created for this activity. For live cloud work, use the Console resource views and activity evidence to verify deletion. Do not delete a shared project, dataset, bucket, registry or endpoint unless the trainer identifies it as disposable.